

Advanced Database Systems Project Report

Business Scenario: Smart Ride System (Ride-Hailing Management)

Submission Date: June 5, 2026

Name	ID	Role
Habiba Hamdy Ali	23011064	Database design, normalization (3NF), and SQL Server implementation.
Mostafa EL-Hosseney Mostafa	23011539	Core implementation, queries, stored procedures, window functions, and analytical (DW) queries.
Habiba Samy Mohammed	23010137	Transactions, concurrency control (simulation & conflicts), and recovery (rollback & failure handling).
Basem Tarek	23010111	Query optimization (indexing & tuning) and MongoDB implementation (CRUD operations).
Habiba Yahia	23011066	Data warehouse design (Star Schema), ETL process, and data population.

1. Database Design & Normalization

The project implements a **Smart Ride System** designed to manage users, drivers, vehicle fleets, and trip transactions. Based on the provided EERD and Normalization documents:

- **EERD:** Captures complex relationships such as User-Trip (One-to-Many), Driver-Vehicle (One-to-One/Many), and Trip-Payment.
- **Normalization:** All tables have been normalized to the **Third Normal Form (3NF)**. For example, the `Trips` table separates payment details and driver info to eliminate transitive dependencies and data redundancy.

2. Implementation & Core Logic

2.1 Stored Procedures

A specialized stored procedure `BookTrip` was implemented to automate the ride-booking process. It validates driver availability, creates a trip record, and updates the driver status in a single atomic operation.

2.2 Transactions & Recovery

To ensure data integrity during payment processing, **SQL Transactions** are used. The system demonstrates a `ROLLBACK` mechanism where if a payment gateway fails, the trip status is reverted to 'Pending' to prevent inconsistent states (System Failure Simulation).

3. Concurrency Control

We simulated concurrent access using two SQL sessions attempting to update the same `DriverID` status simultaneously. The system handles this using **Isolation Levels**, resulting in a temporary **Blocking** state until the first transaction commits, preventing "Lost Updates."

4. Query Optimization & Analytics

4.1 Indexing Strategy

To improve performance, the following were applied:

- **Clustered Index:** On `TripID` for fast primary key lookups.
- **Non-Clustered Index:** On `UserID` and `PickupTime` to speed up user history queries.

4.2 Window Functions

Advanced analytical queries were written using:

- `ROW_NUMBER()` : To sequence trips for each user.
- `RANK()` : To rank drivers based on their average ratings from the Reviews table.

5. Data Warehouse (Star Schema)

A separate Data Warehouse schema was designed for long-term analytical processing (OLAP). It consists of:

- **Fact Table:** `FactTrips` (containing `TripID`, `Fare`, `Distance`, and Keys).
- **Dimension Tables:** `DimUsers`, `DimDrivers`, and `DimTime`.

This allows for complex trend analysis, such as calculating monthly revenue growth or peak-hour ride frequency.

6. NoSQL Integration (MongoDB)

The `Users` table was mapped to a MongoDB Collection. This allows for a flexible schema where user preferences and multiple contact methods are stored as nested documents. Operations included **Insert** of new profile data, **Find** for specific user filters, and **Update** for profile settings.